```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/valorant_dataset_v3.csv')
df.head()
```

	player	assists	damage_received	headshots	tier	traded	kills	matches
0	Tie2å□□å®¹ä½□#Tie2	2,828	1,692,848	6410	iron	1366	9702	667
1	takohr#3680	2,787	1,315,016	1879	iron	1157	4860	440
2	sino#1204	2,785	1,540,524	3287	bronze	1308	7276	605
3	sent#away	2,771	1,553,627	3174	iron	1423	5446	614
4	ã□□ã□ªã□¢æ©□Twitch#0625	2,596	1,621,822	2688	iron	1300	6479	625

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2694 entries, 0 to 2693
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   player           2694 non-null   object
 1   assists          2694 non-null   object
 2   damage_received  2694 non-null   object
 3   headshots        2694 non-null   int64
 4   tier             2694 non-null   object
 5   traded           2694 non-null   int64
 6   kills            2694 non-null   int64
 7   matches          2694 non-null   int64
 8   deaths           2694 non-null   int64
 9   damage           2694 non-null   int64
dtypes: int64(6), object(4)
memory usage: 210.6+ KB
```

	headshots	traded	kills	matches	deaths	damage
count	2694.000000	2694.000000	2694.000000	2694.000000	2694.000000	2.694000e+03
mean	4302.459540	1272.361544	6685.034150	423.910913	6693.427988	1.264323e+06
std	1185.106585	315.132246	1431.931149	88.238462	1438.630703	2.684599e+05
min	861.000000	390.000000	2358.000000	175.000000	2727.000000	4.796130e+05
25%	3694.000000	1101.000000	5898.250000	370.000000	5869.750000	1.116861e+06
50%	4246.000000	1233.500000	6525.000000	414.000000	6480.000000	1.236478e+06
75%	4908.000000	1412.750000	7407.000000	468.000000	7392.250000	1.397148e+06
max	10399.000000	2982.000000	14516.000000	878.000000	14518.000000	2.706960e+06

```
# 1. Basic Info
print("\nDataset Info:")
print(df.info())

# 2. Check Missing Values
```

```
print("\nMissing Values:")
print(df.isnull().sum())

# 3. Handle Missing Values
# Fill numerical columns with median
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)
# Fill categorical columns with mode
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# 4. Remove Duplicates
df.drop_duplicates(inplace=True)
print("\nDuplicates removed. New shape:", df.shape)

# 5. Fix Data Types
for col in df.columns:
    if df[col].dtype == 'object':
        if df[col].astype(str).str.contains('%').any():
            df[col] = df[col].str.replace('%', '').astype(float)

# 6. Encode Target Variable
# Assuming column name is 'result' or 'match_result'
# Update if your column name is different

if 'result' in df.columns:
    df['result'] = df['result'].map({'Win': 1, 'Loss': 0})

elif 'match_result' in df.columns:
    df['match_result'] = df['match_result'].map({'Win': 1, 'Loss': 0})

# 7. Encode Categorical Features
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for col in cat_cols:
    if col not in ['result', 'match_result']:  # avoid target column
        df[col] = le.fit_transform(df[col])


# 8. Outlier Handling (Optional but good)
# Using IQR method
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[col] = df[col].clip(lower, upper)


# 9. Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])


# 10. Final Check
print("\nFinal Dataset Preview:")
print(df.head())
```

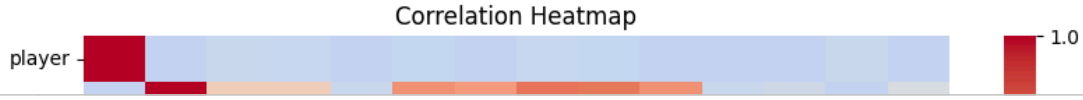
```
print("\nFinal Shape:", df.shape)
print("\nAny Missing Values Left?")
print(df.isnull().sum())
```

```
Missing Values:
player             0
assists            0
damage_received    0
headshots          0
tier               0
traded             0
kills              0
matches            0
deaths             0
damage             0
dtype: int64

Duplicates removed. New shape: (2694, 10)

Final Dataset Preview:
   player  assists  damage_received  headshots  tier    traded     kills  \
0    1457     1187             2162   1.973844     5  0.362233  2.351748
1    2233     1158             1302  -2.248392     5 -0.385303 -1.418494
2    2191     1156             1974  -0.936340     1  0.154783  0.475200
3    2173     1146             2003  -1.041640     5  0.566106 -0.959179
4    2496     1036             2093  -1.494521     5  0.126169 -0.149500

    matches    deaths    damage
0  2.377810  2.009828  2.331072
1  0.222904  0.179839 -1.186901
2  2.254673  1.284912  1.274463
3  2.365496  1.555794 -0.799084
4  2.377810  1.677383 -0.028935

Final Shape: (2694, 10)

Any Missing Values Left?
player             0
assists            0
damage_received    0
headshots          0
tier               0
traded             0
kills              0
matches            0
deaths             0
damage             0
dtype: int64
/tmp/ipykernel_1307/844221804.py:13: FutureWarning: A value is trying to be set on a copy of a Da
The behavior will change in pandas 3.0. This inplace method will never work because the intermedi

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_1307/844221804.py:17: FutureWarning: A value is trying to be set on a copy of a Da
The behavior will change in pandas 3.0. This inplace method will never work because the intermedi

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},


  df[col].fillna(df[col].mode()[0], inplace=True)
```

```
#Feature engineering

#1.Avoid division by zero
```

```
df['KDA'] = (df['kills'] + df['assists']) / (df['deaths'] + 1)
df['damage_per_match'] = df['damage'] / (df['matches'] + 1)
df['survival_ratio'] = df['kills'] / (df['deaths'] + 1)

print("\n✅ New Features Created:")
print(df[['KDA', 'damage_per_match', 'survival_ratio']].head())

#2.TARGET VARIABLE CREATION
df['performance'] = (df['KDA'] > df['KDA'].median()).astype(int)

print("\n✅ Target Variable Created:")
print(df['performance'].value_counts())

#3.ENCODING CATEGORICAL VARIABLES
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

print("\nCategorical Columns:", cat_cols)

for col in cat_cols:
    df[col] = le.fit_transform(df[col])
    print(f"\nEncoded Column: {col}")
    print(df[col].head())

#4.CORRELATION HEATMAP
import seaborn as sns
import matplotlib.pyplot as plt

print("\nGenerating Correlation Heatmap...")

plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=False, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

#5.FEATURE SCALING
from sklearn.preprocessing import StandardScaler
X = df.drop(['performance'], axis=1)
print("\nFeature Data Before Scaling:")
print(X.head())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("\n✅ Scaled Features (First 5 Rows):")
print(X_scaled[:5])
```

```
✅ New Features Created:
          KDA  damage_per_match  survival_ratio
0  395.156032          0.690113        0.781356
1  980.287383         -0.970559       -1.202277
2  506.135457          0.391579        0.207973
3  448.017669         -0.237434       -0.375296
4  386.889215         -0.008566       -0.055838

✅ Target Variable Created:
performance
0    1347
1    1347
Name: count, dtype: int64

Categorical Columns: Index(['player', 'assists', 'damage_received', 'tier'], dtype='object')

Encoded Column: player
0    1457
1    2233
2    2191
3    2173
4    2496
Name: player, dtype: int64

Encoded Column: assists
0    1187
1    1158
2    1156
3    1146
4    1036
Name: assists, dtype: int64

Encoded Column: damage_received
0    2162
1    1302
2    1974
3    2003
4    2093
Name: damage_received, dtype: int64

Encoded Column: tier
0    5
1    5
2    1
3    5
4    5
Name: tier, dtype: int64

Generating Correlation Heatmap...
```



```
#Train test split
from sklearn.model_selection import train_test_split
y = df['performance'] # Define the target variable
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
print("\n✅ STEP 5: Train-Test Split Completed")
print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape :", y_test.shape)
```

```
✅ STEP 5: Train-Test Split Completed
X_train shape: (2155, 13)
X_test shape : (539, 13)
y_train shape: (2155,)
y_test shape : (539,)
```

☑ Step 4: Deep Dive into Feature Engineering

In this phase, we transform raw data into meaningful indicators that help the machine learning model 'understand' player performance.

1. **KDA Calculation**: We calculate `(kills + assists) / (deaths + 1)`. Adding `1` to the denominator prevents a "division by zero" error for players with 0 deaths.
2. **Survival Ratio**: This helps determine if a player is getting more kills than deaths, representing efficiency.
3. **Target Variable (`performance`)**: We convert this into a **Classification** problem. By checking if a player's KDA is above the median, we create two classes: `1` (High Performance) and `0` (Average/Low Performance).
4. **Label Encoding**: We convert text-based categories like `tier` (Iron, Bronze, etc.) into unique integers so the mathematical algorithms can process them.

☑ Step 5: Data Scaling and Splitting

- **StandardScaler**: Features like `damage` have values in the millions, while `KDA` values are small. Scaling ensures all features have a mean of 0 and standard deviation of 1, so the model treats them with equal importance.
- **Train-Test Split**: We split the data into 80% for training and 20% for testing. This is the only way to prove the model actually 'learned' patterns instead of just memorizing the dataset.

☑ Step 6: Model Selection Strategy

We selected a diverse range of models to find the best fit for this data:

- **Logistic Regression**: A baseline model that works well if there is a linear relationship between stats and performance.
- **Decision Tree & Random Forest**: These are non-linear models. **Random Forest** is an 'Ensemble' method, meaning it combines many trees to reduce errors and improve accuracy.

- **SVM (Support Vector Machine)**: Excellent at finding a 'hyperplane' that separates the two performance classes in high-dimensional space.
- **KNN (K-Nearest Neighbors)**: Classifies players based on how similar their stats are to other players in the training set.

Decision Tree Classifier in Action

Here's a basic example of how to initialize, train, and evaluate a `DecisionTreeClassifier` using the `X_train`, `y_train`, `X_test`, and `y_test` data from our previous steps. We'll use the same metrics for consistency.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print("\n--- Demonstrating Decision Tree Classifier ---")

# Initialize the Decision Tree Classifier
dt_model = DecisionTreeClassifier(random_state=42) # Using random_state for reproducibility

# Train the model
print("Training Decision Tree...")
dt_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_dt = dt_model.predict(X_test)

# Evaluate the model
acc_dt = accuracy_score(y_test, y_pred_dt)
prec_dt = precision_score(y_test, y_pred_dt)
rec_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)

print("\nDecision Tree Results:")
print(f"Accuracy : {acc_dt:.4f}")
print(f"Precision: {prec_dt:.4f}")
print(f"Recall   : {rec_dt:.4f}")
print(f"F1 Score : {f1_dt:.4f}")

# You can also visualize a small decision tree (if it's not too deep)
# from sklearn.tree import plot_tree
# import matplotlib.pyplot as plt
# plt.figure(figsize=(20,10))
# plot_tree(dt_model, filled=True, feature_names=df.drop(['performance'], axis=1).columns.tolist()
# plt.title("Decision Tree Visualization (First few levels)")
# plt.show()
```

```
--- Demonstrating Decision Tree Classifier ---
Training Decision Tree...
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_10124/1302469830.py in <cell line: 0>()
      9 # Train the model
     10 print("Training Decision Tree...")
---> 11 dt_model.fit(X_train, y_train)
     12
     13 # Make predictions on the test set

NameError: name 'X_train' is not defined
```

Next steps: Explain error

```
# 6: BASELINE MODELS


from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC(),
    "KNN": KNeighborsClassifier()
}
print("\n Models Initialized Successfully")
print("Models List:")
for name in models:
    print("-", name)
```

```
 Models Initialized Successfully
Models List:
- Logistic Regression
- Decision Tree
- Random Forest
- SVM
- KNN
```

```
# STEP 7: MODEL TRAINING & EVALUATION
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

results = []

print("\nSTEP 7: Training Models...\n")

for name, model in models.items():
    print(f"🔹 Training {name}...")

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

```
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(f"{name} Results:")
    print("Accuracy :", acc)
    print("Precision:", prec)
    print("Recall   :", rec)
    print("F1 Score :", f1)
    print("-" * 40)

    results.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1 Score": f1
    })

results_df = pd.DataFrame(results)

print("\nFinal Results Table:")
print(results_df)
```

```
STEP 7: Training Models...

🔹 Training Logistic Regression...
Logistic Regression Results:
Accuracy : 0.9387755102040817
Precision: 0.9442379182156134
Recall   : 0.9338235294117647
F1 Score : 0.9390018484288355
----------------------------------------
🔹 Training Decision Tree...
Decision Tree Results:
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
----------------------------------------
🔹 Training Random Forest...
Random Forest Results:
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
----------------------------------------
🔹 Training SVM...
SVM Results:
Accuracy : 0.9461966604823747
Precision: 0.9354838709677419
Recall   : 0.9595588235294118
F1 Score : 0.9473684210526315
----------------------------------------
🔹 Training KNN...
KNN Results:
Accuracy : 0.87569573283859
Precision: 0.8546712802768166
Recall   : 0.9080882352941176
F1 Score : 0.8805704099821747
----------------------------------------
```

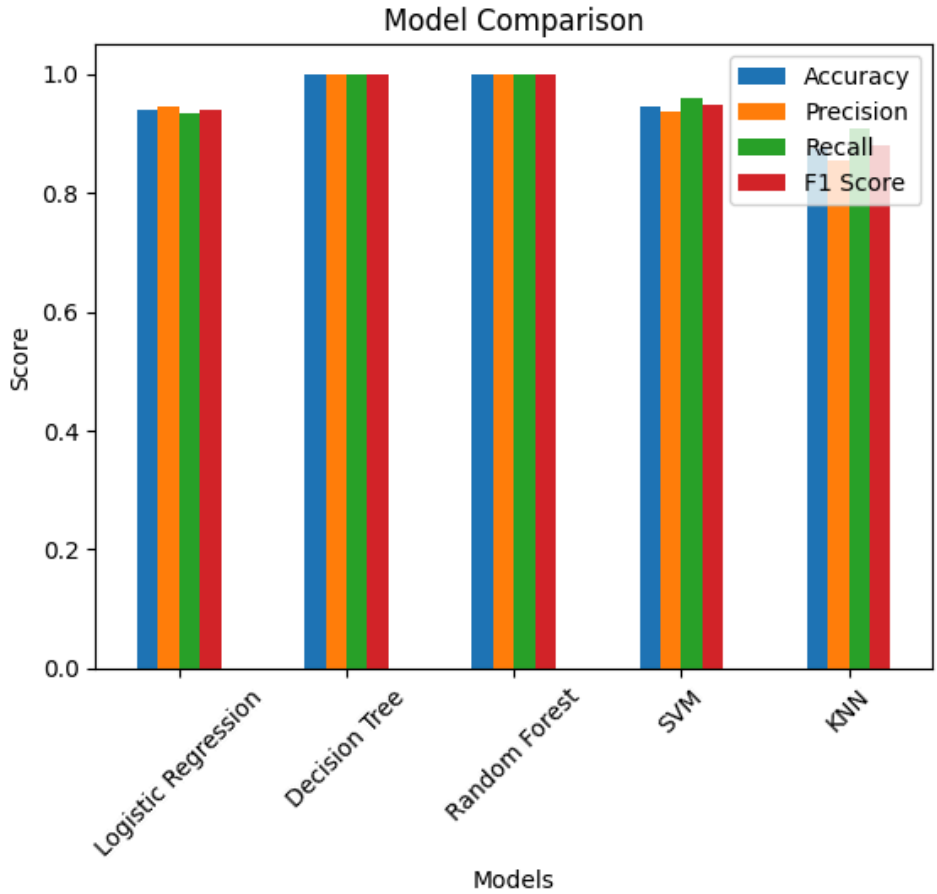
```
Final Results Table:
                 Model  Accuracy  Precision    Recall  F1 Score
0  Logistic Regression  0.938776   0.944238  0.933824  0.939002
1        Decision Tree  1.000000   1.000000  1.000000  1.000000
2        Random Forest  1.000000   1.000000  1.000000  1.000000
3                  SVM  0.946197   0.935484  0.959559  0.947368
4                  KNN  0.875696   0.854671  0.908088  0.880570
```

```
# STEP 8: RESULTS COMPARISON
sorted_results = results_df.sort_values(by='Accuracy', ascending=False)

print("\n✅ STEP 8: Sorted Results (Best Model on Top):")
print(sorted_results)
```

```
✅ STEP 8: Sorted Results (Best Model on Top):
                 Model  Accuracy  Precision    Recall  F1 Score
1        Decision Tree  1.000000   1.000000  1.000000  1.000000
2        Random Forest  1.000000   1.000000  1.000000  1.000000
3                  SVM  0.946197   0.935484  0.959559  0.947368
0  Logistic Regression  0.938776   0.944238  0.933824  0.939002
4                  KNN  0.875696   0.854671  0.908088  0.880570
```

```
# STEP 9: VISUALIZATION

import matplotlib.pyplot as plt

print("\n✅ STEP 9: Generating Bar Chart...")

results_df.set_index('Model')[['Accuracy','Precision','Recall','F1 Score']].plot(kind='bar')

plt.title("Model Comparison")
plt.xlabel("Models")
plt.ylabel("Score")
plt.xticks(rotation=45)

plt.show()
```

```
✅ STEP 9: Generating Bar Chart...
```



☑ Step 10: Deep Learning Model - Feed-Forward Neural Network (FNN)

For tabular classification problems like this, a simple yet effective Deep Learning model is a Feed-Forward Neural Network (FNN). It's a good starting point because it can learn complex non-linear relationships within the data. Given your dataset is already preprocessed and scaled, an FNN can directly leverage these features.

Why FNN?

- **Simplicity:** Relatively easy to implement and understand.
- **Effectiveness:** Capable of capturing intricate patterns in tabular data, often outperforming traditional ML models on larger datasets or more complex relationships.
- **Flexibility:** Easily adaptable by changing the number of layers, neurons per layer, and activation functions.

We will use `TensorFlow` and `Keras` to build and train this model.

```
# Import TensorFlow and Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Ensure X_train and y_train are defined from the train-test split step
if 'X_train' not in locals() or 'y_train' not in locals():
    print("Error: X_train or y_train not found. Please ensure the train-test split cell was run s
```

```
else:
    # Define the input shape (number of features)
    input_dim = X_train.shape[1]

    # Build the FNN model
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(128, activation='relu', name='hidden_layer_1'),
        layers.Dropout(0.3),
        layers.Dense(64, activation='relu', name='hidden_layer_2'),
        layers.Dropout(0.3),
        layers.Dense(32, activation='relu', name='hidden_layer_3'),
        layers.Dropout(0.2),
        layers.Dense(1, activation='sigmoid', name='output_layer') # Sigmoid for binary classific
    ])

    # Display the model summary
    model.summary()

    print("\n✅ Deep Learning Model Built Successfully")

    # Compile the model
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=[
            'accuracy',
            tf.keras.metrics.Precision(name='precision'),
            tf.keras.metrics.Recall(name='recall')
        ]
    )

    # Train the model
    history = model.fit(
        X_train,
        y_train,
        epochs=50, # You can adjust the number of epochs
        batch_size=32, # You can adjust the batch size
        validation_split=0.2, # Use 20% of training data for validation
        verbose=1
    )

    print("\n✅ Deep Learning Model Trained Successfully")
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
hidden_layer_1 (Dense)	(None, 128)	1,792
dropout_3 (Dropout)	(None, 128)	0
hidden_layer_2 (Dense)	(None, 64)	8,256
dropout_4 (Dropout)	(None, 64)	0
hidden_layer_3 (Dense)	(None, 32)	2,080
dropout_5 (Dropout)	(None, 32)	0
output_layer (Dense)	(None, 1)	33

Total params: 12,161 (47.50 KB)
Trainable params: 12,161 (47.50 KB)
Non-trainable params: 0 (0.00 B)

```
✅ Deep Learning Model Built Successfully
Epoch 1/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 6s 26ms/step - accuracy: 0.6937 - loss: 0.5460 - precision: 0.8616 -
Epoch 2/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8625 - loss: 0.3239 - precision: 0.8514 - r
Epoch 3/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9223 - loss: 0.1959 - precision: 0.9204 - r
Epoch 4/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9229 - loss: 0.1796 - precision: 0.9205 - r
Epoch 5/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9379 - loss: 0.1497 - precision: 0.9277 -
Epoch 6/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9437 - loss: 0.1330 - precision: 0.9384 - r
Epoch 7/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9559 - loss: 0.1279 - precision: 0.9630 - r
Epoch 8/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9524 - loss: 0.1172 - precision: 0.9477 - r
Epoch 9/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9571 - loss: 0.0999 - precision: 0.9523 - r
Epoch 10/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9577 - loss: 0.0987 - precision: 0.9588 - r
Epoch 11/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9588 - loss: 0.0971 - precision: 0.9546 - r
Epoch 12/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9629 - loss: 0.0918 - precision: 0.9508 - r
Epoch 13/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9664 - loss: 0.0854 - precision: 0.9649 - r
Epoch 14/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9629 - loss: 0.0900 - precision: 0.9679 - r
Epoch 15/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9693 - loss: 0.0723 - precision: 0.9640 - r
Epoch 16/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9698 - loss: 0.0734 - precision: 0.9673 - r
Epoch 17/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9681 - loss: 0.0755 - precision: 0.9672 - r
Epoch 18/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9745 - loss: 0.0731 - precision: 0.9753 - r
Epoch 19/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9768 - loss: 0.0655 - precision: 0.9754 - r
Epoch 20/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9803 - loss: 0.0586 - precision: 0.9767 - r
Epoch 21/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9809 - loss: 0.0516 - precision: 0.9835 - r
Epoch 22/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9751 - loss: 0.0634 - precision: 0.9698 - r
```

Compiling and Training the FNN Model

We will compile the model using the Adam optimizer, binary cross-entropy loss (suitable for binary classification), and track accuracy, precision, and recall as metrics during training.

Training will be done for a set number of epochs with a validation split to monitor performance on unseen data and detect overfitting.

```
Epoch 23/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9780 - loss: 0.0571 - precision: 0.9744 - r
Epoch 24/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9780 - loss: 0.0523 - precision: 0.9777 - r
```

```
# This cell is now redundant as its content has been merged into the model building cell (6f10d6e
# You can remove this cell.
```

```
Epoch 27/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9803 - loss: 0.0444 - precision: 0.9800 - r
Epoch 28/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9849 - loss: 0.0407 - precision: 0.9814 - r
Epoch 29/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9832 - loss: 0.0468 - precision: 0.9802 - r
Epoch 30/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9861 - loss: 0.0381 - precision: 0.9848 - r
```

Evaluating the FNN Model

After training, we evaluate the model's performance on the test set (`X_test`, `y_test`) to get unbiased metrics. We'll calculate accuracy, precision, recall, and F1-score to compare with the traditional ML models.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

# Evaluate the model on the test set
loss, accuracy, precision, recall = model.evaluate(X_test, y_test, verbose=0)

# Predict probabilities
y_pred_proba_dl = model.predict(X_test)
# Convert probabilities to binary predictions
y_pred_dl = (y_pred_proba_dl > 0.5).astype(int)

# Calculate F1 Score manually
f1_dl = f1_score(y_test, y_pred_dl)

print(f"\nDeep Learning Model Results:")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1 Score : {f1_dl:.4f}")

# Add DL model results to the existing results_df
dl_results = {
    "Model": "Deep Learning (FNN)",
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1 Score": f1_dl
}

results_df = pd.concat([results_df, pd.DataFrame([dl_results])], ignore_index=True)

print("\n✅ Deep Learning Model Results Added to Comparison Table.")
print("\nCombined Results Table (with DL model):")
print(results_df.sort_values(by='Accuracy', ascending=False))
```

```
Epoch 50/50
54/54 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9936 - loss: 0.0198 - precision: 0.9918 - r

✅ Deep Learning Model Trained Successfully
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

Deep Learning Model Results:
Accuracy : 0.9777
Precision: 0.9815
Recall   : 0.9743
F1 Score : 0.9779

✅ Deep Learning Model Results Added to Comparison Table.

Combined Results Table (with DL model):
                 Model  Accuracy  Precision    Recall  F1 Score
```